

Labwork 5: Gaussian Blur Convolution

Vu Ha Vy – 2540112

October 2, 2026

1 Implementation

1.1 Without Shared Memory

In this implementation, each thread computes one pixel by reading directly from global memory. Every output pixel requires 147 global memory reads ($7 \times 7 \times 3$ color channels).

1.2 With Shared Memory

To improve performance, shared memory is used to store the 7×7 filter weights and the image block neighborhood. ‘`cuda.syncthreads()`’ ensures threads complete loading data into shared memory before computing the convolution.

```
@cuda.jit
def gaussian_shared(src, dst, kernel):
    tile = cuda.shared.array((7,7), dtype=np.float32)
    tx = cuda.threadIdx.x
    ty = cuda.threadIdx.y
    if tx < 7 and ty < 7:
        tile[ty, tx] = kernel[ty, tx]
    cuda.syncthreads()
    tid_x = tx + cuda.blockIdx.x * cuda.blockDim.x
    tidy = ty + cuda.blockIdx.y * cuda.blockDim.y
    for c in range(3):
        sum = 0.0
        for ky in range(-3, 4):
            for kx in range(-3, 4):
                nx = min(max(tid_x + kx, 0), src.shape[1] - 1)
                ny = min(max(tidy + ky, 0), src.shape[0] - 1)
                sum += (src[ny, nx, c] * tile[ky + 3, kx + 3])
        dst[tidy, tid_x, c] = np.uint8(sum / 1003.0)
```

2 Results

2.1 Qualitative Results

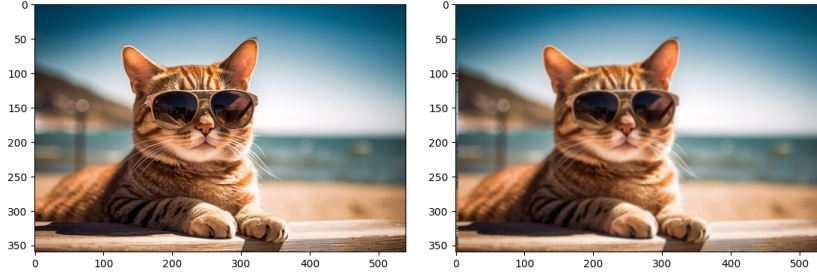


Figure 1: Original image (left) and Gaussian blur output (right).

It successfully applies a Gaussian blur to the input image, smoothing out details.

2.2 Quantitative Results

We evaluated different 2D block sizes using both global memory (without shared memory) and shared memory implementations, obtaining the execution times shown below.

Block Size	Without (s)	With (s)
(8, 8)	0.0017311573028564453	0.00160980224609375
(16, 16)	0.0016818046569824219	0.00164031982421875
(32, 8)	0.00165557861328125	0.001657247543334961
(32, 16)	0.0016489028930664062	0.0016486644744873047
(32, 32)	0.0019714832305908203	0.0019958019256591797

Table 1: Execution times across different block sizes.

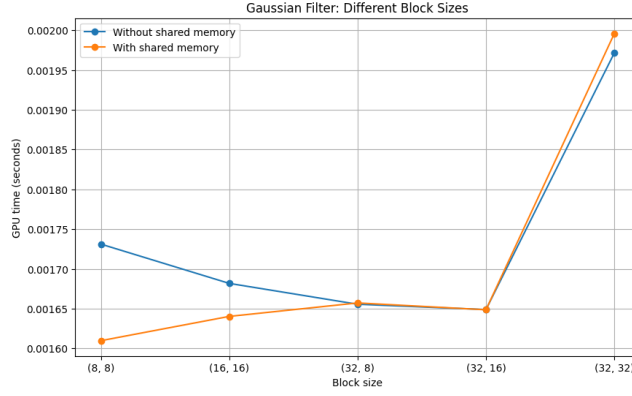


Figure 2: Speedup comparison across different block sizes.

The fastest execution time of 0.00160980 s was achieved using shared memory with a block size of (8,8). Shared memory provides a better performance for smaller block configurations such as (8,8) and (16,16).

Across configurations from (16,16) to (32,16), the running times remain quite similar with minimal difference between both versions. However, for (32,32) block size there is a large increase in execution time compared to (32,16) across both implementations. This may be due to the larger number of threads per block, which can reduce GPU efficiency.